EXPERIMENT-10

TITLE: Views, Materialized Views, Functions, Stored Procedures and Triggers

OBJECTIVE: To understand and implement Views, Materialized Views, Functions, Stored Procedures, and Triggers in SQL.

Aim: To implement database views, materialized views, functions, stored procedures, and triggers using SQL.

Theory:

PL/SQL is Oracle's procedural extension to SQL. Cursors allow row-by-row processing, triggers respond to database events, and procedures/functions encapsulate reusable logic.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE GetEmployee()
BEGIN
  SELECT * FROM employee;
END//

CREATE TRIGGER before_insert_employee
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
  SET NEW.salary = IFNULL(NEW.salary, 10000);
END//
DELIMITER ;
```

Output:

The expected output confirms successful execution of SQL commands and matches the theoretical concept.Result:

Successfully executed as per the aim of the experiment.

Exercise

Create the following two tables:

DEPARTMENT TABLE

DEPT_ID	DEPT_NAME	LOCATION
10	Computer	Ahmedabad
20	Mechanical	Rajkot
30	Electrical	Gandhinagar
40	Civil	Surat

EMPLOYEE TABLE

EMP_ID	EMP_NAME	JOB	SALARY	DEPT_ID
101	Amit	Manager	60000	10
102	Neha	Developer	45000	10
103	Rahul	Developer	40000	10
104	Priya	Analyst	38000	20
105	Karan	Manager	55000	20
106	Riya	Engineer	42000	30
107	Jay	Engineer	35000	30
108	Meera	Analyst	32000	40

Write the SQL queries for the following:

1. Create a view named EMP_DEPT_VIEW displaying employee ID, employee name, job, salary, and department name.

2. Display all the records from EMP_DEPT_VIEW.

3. Using the view, display employees whose salary is greater than 40000.

4. Create a materialized view named DEPT_SALARY_SUMMARY to store department-wise total salary and average salary.

5. Display the records from the materialized view.

6. Create a function ANNUAL_SALARY that accepts monthly salary and returns the annual salary.

7. Display employee name, monthly salary, and annual salary using the function created above.

8. Create a stored procedure GET_EMP_BY_DEPT that accepts department ID and displays all employees belonging to that department.

9. Create a stored procedure INCREASE_SALARY that accepts employee ID and percentage increment and updates the employee's salary.

10. Create a trigger CHECK_SALARY that prevents insertion or updating of an employee when the salary is less than 20000.

11. Create a table SALARY_AUDIT to store employee ID, old salary, new salary, and date/time of salary modification.

12. Create a trigger that automatically stores the old and new salary in SALARY_AUDIT whenever an employee's salary is updated.

13. Test the function, stored procedures, and triggers using suitable SQL statements.

Result: Successfully executed the SQL commands as per the aim of the experiment.